Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Ярмолинский И.В.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Ход работы

Вариант 10 Прогнозирование уровня дохода

- Adult Census Income
- **Задача:** предсказать, превышает ли доход $50 тыс. в год (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 32 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** сравните производительность с более глубокой моделью: два скрытых слоя по 16 нейронов.

Код программы

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import TensorDataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print("Загрузка датасета Adult Census Income...")

columns = [
    "age",
    "workclass",
    "fnlwgt",
    "education",
    "education-num",
    "marital-status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "capital-gain",
    "capital-loss",
    "hours-per-week",
    "native-country",
    "income"
]

# Загрузка обучающей части датасета
train_url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/adult/adult.data"
)
```

```
# Загрузка тестовой части датасета
test_url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/adult/adult.test"
)

train_df = pd.read_csv(
    train_url,
    names=columns,
    skipinitialspace=True,
    na_values="?"
)

test_df = pd.read_csv(
    test_url,
    names=columns,
    skipinitialspace=True,
    na_values="?",
    skiprows=1
)

# Объединение данных
df = pd.concat([train_df, test_df], ignore_index=True)

# Очистка целевого признака
df["income"] = (
    df["income"]
    .astype(str)
    .str.strip()
    .str.replace(".", "", regex=False)
)

# Преобразование дохода в бинарные значения:
# 0 - доход <= 50K
# 1 - доход > 50K

df["income"] = df["income"].map({
    "<=50K": 0,
    ">50K": 1
})

df = df.dropna(subset=["income"])

print("Размер датасета:", df.shape)
print("\nРаспределение классов:")
print(df["income"].value_counts())

# Разделение признаков и целевой переменной
X = df.drop("income", axis=1)
y = df["income"].astype(int)

# Числовые признаки
numeric_features = [
    "age",
    "fnlwgt",
    "education-num",
    "capital-gain",
    "capital-loss",
    "hours-per-week"
]

# Категориальные признаки
categorical_features = [
    "workclass",
    "education",
    "marital-status",
    "occupation",
    "relationship",
    "race",
    "sex",
```

```
    "native-country"
]

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Обработка числовых признаков
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Обработка категориальных признаков
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore",
                              sparse_output=False))
])

# Общая предобработка
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# Обучение преобразователя только на обучающей выборке
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

# Преобразование в numpy-массивы
X_train = np.asarray(X_train, dtype=np.float32)
X_test = np.asarray(X_test, dtype=np.float32)

y_train = y_train.to_numpy(dtype=np.float32).reshape(-1, 1)
y_test = y_test.to_numpy(dtype=np.float32).reshape(-1, 1)

# Преобразование в тензоры PyTorch
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)

y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

print("\nДанные успешно подготовлены!")
print("Размер обучающей выборки:", X_train_tensor.shape)
print("Размер тестовой выборки:", X_test_tensor.shape)

class MLP(nn.Module):

    def __init__(self, input_size, hidden_layers):
        super(MLP, self).__init__()

        layers = []
        previous_size = input_size

        # Создание скрытых слоев
        for neurons in hidden_layers:
            layers.append(nn.Linear(previous_size, neurons))
            layers.append(nn.ReLU())
            previous_size = neurons

        # Выходной слой
        layers.append(nn.Linear(previous_size, 1))

        self.network = nn.Sequential(*layers)
```

```
    def forward(self, x):
        return self.network(x)

# Количество входных признаков
input_size = X_train_tensor.shape[1]

# Первая модель: один скрытый слой из 32 нейронов
model1 = MLP(input_size, [32])

# Вторая модель: два скрытых слоя по 16 нейронов
model2 = MLP(input_size, [16, 16])

def train_model(model, X_train, y_train, epochs=30):

    criterion = nn.BCEWithLogitsLoss()

    optimizer = optim.Adam(
        model.parameters(),
        lr=0.001
    )

    # Создание DataLoader для пакетного обучения
    dataset = TensorDataset(X_train, y_train)

    loader = DataLoader(
        dataset,
        batch_size=128,
        shuffle=True
    )

    print("\nНачало обучения...")

    for epoch in range(epochs):

        model.train()
        total_loss = 0

        for batch_X, batch_y in loader:

            # Прямое распространение
            y_pred = model(batch_X)

            # Расчет ошибки
            loss = criterion(y_pred, batch_y)

            # Обнуление градиентов
            optimizer.zero_grad()

            # Обратное распространение ошибки
            loss.backward()

            # Обновление параметров
            optimizer.step()

            total_loss += loss.item()

        average_loss = total_loss / len(loader)

        if (epoch + 1) % 5 == 0:
            print(
                f"Эпоха [{epoch + 1}/{epochs}], "
                f"Loss: {average_loss:.4f}"
            )

    return model


def evaluate_model(model, X_test, y_test):

    model.eval()
```

```
    with torch.no_grad():

        # Получение логитов
        logits = model(X_test)

        # Применение Sigmoid
        probabilities = torch.sigmoid(logits)

        # Получение классов
        predictions = (probabilities >= 0.5).int()

    y_true = y_test.numpy().flatten()
    y_pred = predictions.numpy().flatten()

    # Расчет метрик
    accuracy = accuracy_score(y_true, y_pred)

    precision = precision_score(
        y_true, y_pred, zero_division=0
    )

    recall = recall_score(
        y_true, y_pred, zero_division=0
    )

    f1 = f1_score(
        y_true, y_pred, zero_division=0
    )

    print("\nРезультаты модели:")
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")

    print("\nМатрица ошибок:")
    print(confusion_matrix(y_true, y_pred))

    print("\nОтчет классификации:")
    print(classification_report(
        y_true,
        y_pred,
        target_names=["Доход <= 50K", "Доход > 50K"],
        zero_division=0
    ))

    return accuracy, precision, recall, f1

torch.manual_seed(42)
np.random.seed(42)

# Обучение первой модели
print("\n==================================")
print("МОДЕЛЬ 1: ОДИН СКРЫТЫЙ СЛОЙ (32)")
print("==================================")

model1 = train_model(
    model1,
    X_train_tensor,
    y_train_tensor,
    epochs=30
)

print("\nОценка первой модели:")
results1 = evaluate_model(
    model1,
    X_test_tensor,
    y_test_tensor
)
```

```
# Обучение второй модели
print("\n==================================")
print("МОДЕЛЬ 2: ДВА СКРЫТЫХ СЛОЯ (16, 16)")
print("==================================")

model2 = train_model(
    model2,
    X_train_tensor,
    y_train_tensor,
    epochs=30
)

print("\nОценка второй модели:")
results2 = evaluate_model(
    model2,
    X_test_tensor,
    y_test_tensor
)

print("\n==================================")
print("СРАВНЕНИЕ МОДЕЛЕЙ")
print("==================================")

comparison = pd.DataFrame({
    "Метрика": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-score"
    ],
    "Модель 1 (32)": results1,
    "Модель 2 (16, 16)": results2
})

print(comparison.to_string(index=False))

if results1[3] > results2[3]:
    print("\nПервая модель показала лучший F1-score.")
elif results2[3] > results1[3]:
    print("\nВторая модель показала лучший F1-score.")
else:
    print("\nОбе модели показали одинаковый F1-score.")

print("\nПрограмма завершена!")
```

Вывод программы

Распределение классов:
income
0 37155
1 11687
Name: count, dtype: int64

Данные успешно подготовлены!
Размер обучающей выборки: torch.Size([39073, 105])
Размер тестовой выборки: torch.Size([9769, 105])

==================================
МОДЕЛЬ 1: ОДИН СКРЫТЫЙ СЛОЙ (32)
==================================

Начало обучения...
Эпоха [5/30], Loss: 0.3081
Эпоха [10/30], Loss: 0.3037

Эпоха [15/30], Loss: 0.3008
Эпоха [20/30], Loss: 0.2981
Эпоха [25/30], Loss: 0.2966
Эпоха [30/30], Loss: 0.2947

Оценка первой модели:

Результаты модели:
Accuracy: 0.8591
Precision: 0.7328
Recall: 0.6476
F1-score: 0.6876

Матрица ошибок:
[[6879 552]
 [824 1514]]

Отчет классификации:
              precision   recall f1-score   support

Доход <= 50K   0.89   0.93   0.91   7431
 Доход > 50K   0.73   0.65   0.69   2338

    accuracy   0.86   9769
   macro avg   0.81   0.79   0.80   9769
weighted avg   0.85   0.86   0.86   9769

==========
МОДЕЛЬ 2: ДВА СКРЫТЫХ СЛОЯ (16, 16)
==========

Начало обучения...
Эпоха [5/30], Loss: 0.3125
Эпоха [10/30], Loss: 0.3067
Эпоха [15/30], Loss: 0.3024
Эпоха [20/30], Loss: 0.2990
Эпоха [25/30], Loss: 0.2971
Эпоха [30/30], Loss: 0.2960

Оценка второй модели:

Результаты модели:
Accuracy: 0.8630
Precision: 0.7585
Recall: 0.6275
F1-score: 0.6868

Матрица ошибок:
[[6964 467]
 [871 1467]]

Отчет классификации:
              precision   recall f1-score   support

Доход <= 50K   0.89   0.94   0.91   7431
 Доход > 50K   0.76   0.63   0.69   2338

accuracy			0.86	9769
macro avg	0.82	0.78	0.80	9769
weighted avg	0.86	0.86	0.86	9769

==========
СРАВНЕНИЕ МОДЕЛЕЙ
==========

Метрика	Модель 1 (32)	Модель 2 (16, 16)
Accuracy	0.859146	0.863036
Precision	0.732817	0.758532
Recall	0.647562	0.627459
F1-score	0.687557	0.686798

Первая модель показала лучший F1-score.

В ходе работы сравнили две нейронные сети: с одним скрытым слоем из 32 нейронов и с двумя скрытыми слоями по 16 нейронов. По результатам сравнения определили, какая модель лучше предсказывает доход выше $50 000 в год.

Вывод: освоил построение, обучение и оценку MLP для бинарной классификации.

В эксперименте сравнил две модели: с одним скрытым слоем из 32 нейронов и с двумя слоями по 16 нейронов. Сравнил их accuracy, precision, recall и F1-score, чтобы определить, какая модель лучше справляется с прогнозированием дохода выше $50 000 в год.